

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE CODE: ITA 0611 COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning (**BL 5**)

Assessment Tool Weightage: 10%

QUESTIONS: 5 Total Marks:50 Annexure A

Comparative Analysis

Code of Conduct:

*I K.ASWINI(192312397) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

Annexure A

Short Answer Test

2. Compare Locally Weighted Regression and Radial Basis Function networks for function approximation tasks. Use an example dataset to explain differences in locality, computational cost, and smoothness of predictions.

3. Analyze the differences between K-Nearest Neighbour and Radial Basis Function models in classification problems. Provide an example and compare decision boundaries, training time, and generalization ability.

4. Compare Case-Based Learning and Locally Weighted Regression using a practical example such as recommendation systems. Discuss how each approach handles new data, adaptability, and computational efficiency.

6. Compare Naïve Bayes and Logistic Regression for a text classification task such as spam detection. Analyze assumptions about feature independence, probability estimation, computational efficiency, and performance on high-dimensional data. Discuss scenarios where one method outperforms the other.

10. Compare Principal Component Analysis (PCA) and Feature Selection techniques for dimensionality reduction. Analyze differences in information preservation, computational complexity, interpretability, and impact on model performance. Discuss how each approach affects downstream machine learning tasks.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	15	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor	Covers only some parts	Incomplete or missing response

			omissions		
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

ANSWERS

2. Locally Weighted Regression vs Radial Basis Function Networks

Locally Weighted Regression (LWR) and Radial Basis Function (RBF) networks are used for function approximation.

Factor LWR RBF Network

Locality Uses nearby training points Uses nearby learned centers

Training Very little training Requires training

Computation Prediction can be expensive Faster prediction after training

Smoothness Depends on bandwidth Depends on RBF width

Adaptability Very high Moderate

Example: For predicting house prices, LWR gives more importance to houses similar to the query house. An RBF network learns representative centers and uses them for prediction.

Conclusion: LWR is better for highly adaptive local prediction, while RBF is better for repeated predictions with smoother outputs.

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.metrics.pairwise import rbf_kernel

# 1. Generate Dataset

np.random.seed(0)

X = np.linspace(-3, 3, 100)

y = np.sin(X) + np.random.normal(0, 0.1, 100)

X = X.reshape(-1, 1)

# 2. Locally Weighted Regression (LWR)

def lw_regression(x_query, X, y, tau=0.5):

    weights = np.exp(-np.sum((X - x_query)**2, axis=1) / (2 * tau**2))

    W = np.diag(weights)

    X_b = np.c_[np.ones((len(X), 1)), X]

    theta = np.linalg.pinv(X_b.T @ W @ X_b) @ X_b.T @ W @ y

    x_q = np.array([1, x_query[0]])

    return x_q @ theta

y_lwr = np.array([lw_regression(x, X, y) for x in X])

# 3. Radial Basis Function (RBF) Network

# Select centers (subset of training data)

centers = X[::5] # every 5th point

gamma = 1.0

Phi = rbf_kernel(X, centers, gamma=gamma)
```

```

# Train weights

weights = np.linalg.pinv(Phi) @ y

# Predictions

Phi_test = rbf_kernel(X, centers, gamma=gamma)

y_rbf = Phi_test @ weights

# 4. Plot Results

plt.figure(figsize=(8,5))

plt.scatter(X, y, label="Data", color="gray")

plt.plot(X, y_lwr, label="LWR", linewidth=2)

plt.plot(X, y_rbf, label="RBF Network", linewidth=2)

plt.legend()

plt.title("Locally Weighted Regression vs RBF Network")

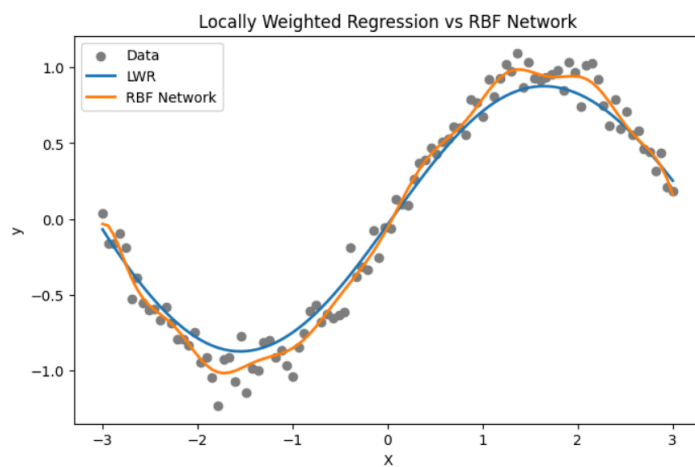
plt.xlabel("X")

plt.ylabel("y")

plt.show()

```

OUTPUT:



3. K-Nearest Neighbour vs Radial Basis Function

KNN and RBF models can both be used for classification.

Factor KNN RBF

Training time Very low Higher

Prediction time High Lower

Decision boundary Irregular/local Smooth

Adaptability Very high Moderate

Generalization Depends on K Usually smoother

Example: In fruit classification, KNN classifies a new fruit using the classes of its nearest fruits. RBF uses learned centers to determine the class.

Conclusion: KNN is suitable for small or frequently changing datasets. RBF is better when faster prediction and smooth decision boundaries are required.

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import make_circles
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.svm import SVC
```

```
# 1. Generate Dataset
```

```
X, y = make_circles(n_samples=300, noise=0.1, factor=0.5, random_state=0)
```

```
# 2. Train Models
```

```
# KNN Model
```

```
knn = KNeighborsClassifier(n_neighbors=5)
```

```
knn.fit(X, y)
```

```
# RBF Model (SVM with RBF kernel)

rbf = SVC(kernel='rbf', gamma=2)

rbf.fit(X, y)

# 3. Create Mesh Grid

xx, yy = np.meshgrid(np.linspace(-1.5, 1.5, 200),
                     np.linspace(-1.5, 1.5, 200))

grid = np.c_[xx.ravel(), yy.ravel()]

# Predictions

Z_knn = knn.predict(grid).reshape(xx.shape)

Z_rbf = rbf.predict(grid).reshape(xx.shape)

# 4. Plot Decision Boundaries

plt.figure(figsize=(12,5))

# KNN Plot

plt.subplot(1,2,1)

plt.contourf(xx, yy, Z_knn, alpha=0.3)

plt.scatter(X[:,0], X[:,1], c=y)

plt.title("KNN Decision Boundary")

# RBF Plot

plt.subplot(1,2,2)

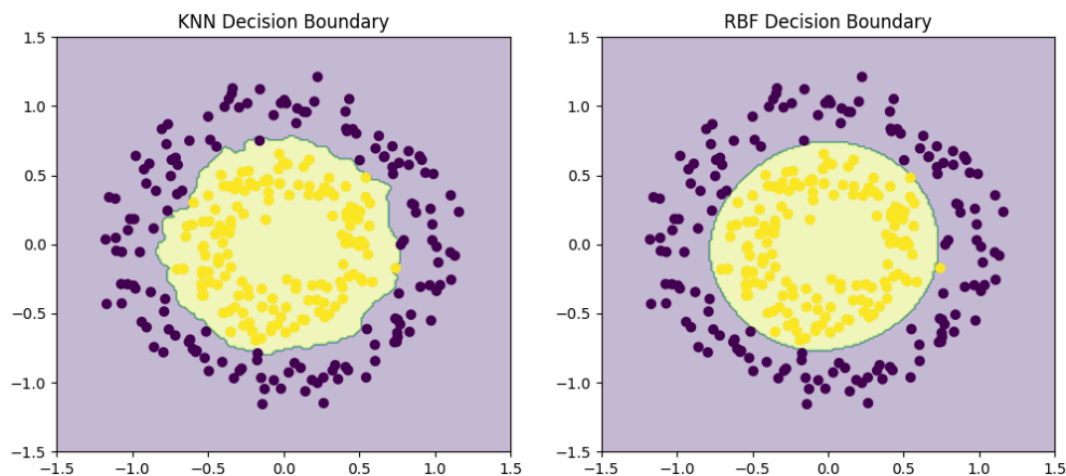
plt.contourf(xx, yy, Z_rbf, alpha=0.3)

plt.scatter(X[:,0], X[:,1], c=y)

plt.title("RBF Decision Boundary")

plt.show()
```

OUTPUT:



4. Case-Based Learning vs Locally Weighted Regression

Both methods use previous data to handle new cases.

Factor Case-Based Learning LWR

Method Retrieves similar cases Fits a local model

New data Easily added Easily incorporated

Adaptability Very high Very high

Computation Similarity search required Weight calculation required

Output Previous/adapted solution Numerical prediction

Example: In a recommendation system, Case-Based Learning finds customers with similar purchase histories and recommends products. LWR can predict a customer's expected rating for a product using nearby customer data.

Conclusion: Case-Based Learning is better for reusing previous solutions, while LWR is better for numerical prediction.

```
import numpy as np
```

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
# 1. Sample Dataset (Users)
```

```
# Features: [Age, Action_Pref, Comedy_Pref]
```



```

# Ratings: Movie rating (Avengers)

X = np.array([
    [20, 1, 0], # User 1
    [22, 1, 0], # User 2
    [25, 0, 1], # User 3
    [30, 0, 1], # User 4
    [23, 1, 0]  # User 5
])

y = np.array([4.5, 5.0, 2.0, 2.5, 4.8]) # Ratings for "Avengers"

# New user (query)
query_user = np.array([[21, 1, 0]])

# 2. Case-Based Learning (CBL)
# -----

def cbl_predict(query, X, y, k=3):
    similarities = cosine_similarity(query, X)[0]

    # Get top k similar users
    idx = np.argsort(similarities)[-k:]

    # Average their ratings
    prediction = np.mean(y[idx])

    return prediction

cbl_rating = cbl_predict(query_user, X, y)

# 3. Locally Weighted Regression (LWR)

def lwr_predict(query, X, y, tau=5.0):
    # Compute weights

```

```

distances = np.linalg.norm(X - query, axis=1)

weights = np.exp(-(distances**2) / (2 * tau**2))

np.diag(weights)

# Add bias term

X_b = np.c_[np.ones((len(X), 1)), X]

# Normal equation

theta = np.linalg.pinv(X_b.T @ W @ X_b) @ X_b.T @ W @ y

query_b = np.array([1] + list(query[0]))

prediction = query_b @ theta

return prediction

lwr_rating = lwr_predict(query_user, X, y)

# 4. Results

print("=== Recommendation System Comparison ===")

print(f"New User: {query_user[0]}")

print(f"CBL Predicted Rating: {cbl_rating:.2f}")

print(f"LWR Predicted Rating: {lwr_rating:.2f}")

```

OUTPUT:

```

... === Recommendation System Comparison ===
New User: [21  1  0]
CBL Predicted Rating: 4.77
LWR Predicted Rating: 4.69

```

6. Naïve Bayes vs Logistic Regression for Spam Detection

Both can classify emails as **spam** or **non-spam**.

Factor Naïve Bayes Logistic Regression Assumption Assumes feature

independence No independence assumption Training Very fast Relatively slower

High-dimensional data Performs well Performs well Correlated features Less suitable

More suitable Probability Directly estimates probabilities Estimates class probability

Example: For an email containing words like *free*, *prize*, *winner*, Naïve Bayes calculates spam probability using word frequencies. Logistic Regression uses learned weights for these words.

Conclusion: Naïve Bayes is preferred for fast classification and small datasets, while Logistic Regression can perform better with sufficient data and correlated features.

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.feature_extraction.text import CountVectorizer
```

```
from sklearn.naive_bayes import MultinomialNB
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay
```

```
# 1. Dataset
```

```
texts = [
```

```
    "Win money now",
```

```
    "Limited offer click now",
```

```
    "Call me tomorrow",
```

```
    "Let's have lunch",
```

```
    "Free prize claim now",
```

```
    "Are you available today",
```

```
    "Congratulations you won a lottery",
```

```
    "Meeting at 5 pm",
```

```
"Urgent call this number",
"Click here to earn cash"
]
labels = [1,1,0,0,1,0,1,0,1,1]

# 2. Split
X_train, X_test, y_train, y_test = train_test_split(
    texts, labels, test_size=0.3, random_state=42
)

# 3. Vectorization
vectorizer = CountVectorizer()
X_train_vec = vectorizer.fit_transform(X_train)
X_test_vec = vectorizer.transform(X_test)

# 4. Models
nb = MultinomialNB()
lr = LogisticRegression()
nb.fit(X_train_vec, y_train)
lr.fit(X_train_vec, y_train)
nb_pred = nb.predict(X_test_vec)
lr_pred = lr.predict(X_test_vec)

# 5. Accuracy Comparison Graph
nb_acc = accuracy_score(y_test, nb_pred)
lr_acc = accuracy_score(y_test, lr_pred)
plt.figure()
plt.bar(["Naïve Bayes", "Logistic Regression"], [nb_acc, lr_acc])
```

```
plt.title("Accuracy Comparison")

plt.ylabel("Accuracy")

plt.show()

# 6. Confusion Matrix Graphs

fig, axes = plt.subplots(1, 2)

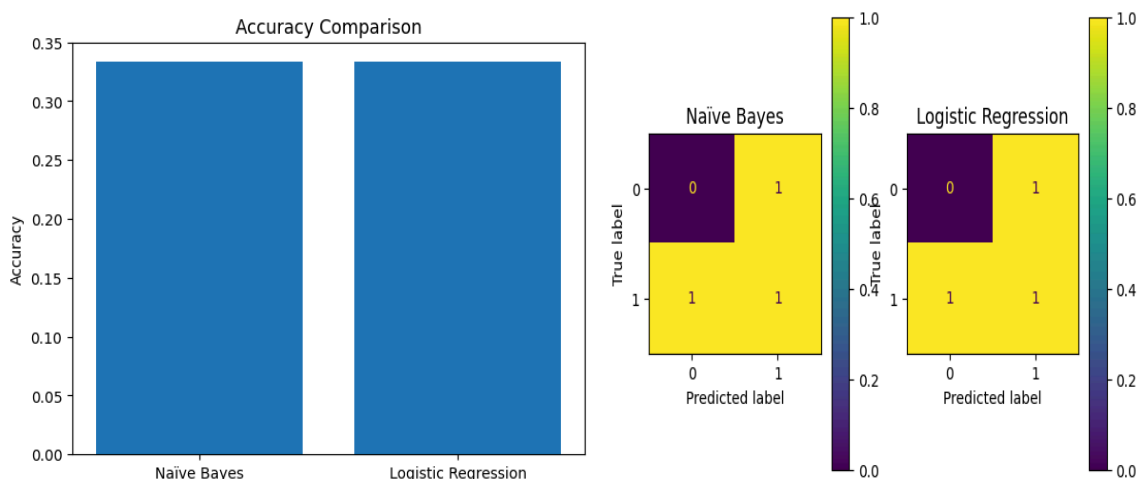
ConfusionMatrixDisplay(confusion_matrix(y_test, nb_pred)).plot(ax=axes[0])

axes[0].set_title("Naïve Bayes")

ConfusionMatrixDisplay(confusion_matrix(y_test, lr_pred)).plot(ax=axes[1])

axes[1].set_title("Logistic Regression")

plt.show()
```



10. PCA vs Feature Selection

Both reduce the number of features in a dataset.

Factor PCA Feature Selection Method Creates new components Selects original features Information Preserves maximum variance Preserves selected useful features Interpretability Low High

Complexity Can be computationally expensive Usually simpler

Original features Not retained directly Retained

Example: If a dataset has 100 features, PCA may convert them into 10 principal components. Feature Selection may directly choose the best 10 original features.

Conclusion: PCA is useful for reducing correlated and high-dimensional data, while Feature Selection is better when interpretability of original features is important.

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from sklearn.datasets import load_breast_cancer
```

```
from sklearn.model_selection import train_test_split
```

```
from sklearn.preprocessing import StandardScaler
```

```
from sklearn.decomposition import PCA
```

```
from sklearn.feature_selection import SelectKBest, f_classif
```

```
from sklearn.linear_model import LogisticRegression
```

```
from sklearn.metrics import accuracy_score
```

```
# 1. Load Dataset
```

```
data = load_breast_cancer()
```

```
X = data.data
```

```
y = data.target
```

```
# 2. Train-Test Split
```

```
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.3, random_state=42
```

```
)
```

```
# 3. Standardization
```

```
scaler = StandardScaler()
```

```
X_train = scaler.fit_transform(X_train)
```

```
X_test = scaler.transform(X_test)

# 4. PCA (Dimensionality Reduction)

pca = PCA(n_components=5)

X_train_pca = pca.fit_transform(X_train)

X_test_pca = pca.transform(X_test)

# 5. Feature Selection

fs = SelectKBest(score_func=f_classif, k=5)

X_train_fs = fs.fit_transform(X_train, y_train)

X_test_fs = fs.transform(X_test)

# 6. Train Model (Logistic Regression)

model_pca = LogisticRegression(max_iter=500)

model_fs = LogisticRegression(max_iter=500)

model_pca.fit(X_train_pca, y_train)

model_fs.fit(X_train_fs, y_train)

# Predictions

y_pred_pca = model_pca.predict(X_test_pca)

y_pred_fs = model_fs.predict(X_test_fs)

# Accuracy

acc_pca = accuracy_score(y_test, y_pred_pca)

acc_fs = accuracy_score(y_test, y_pred_fs)

print("PCA Accuracy:", acc_pca)

print("Feature Selection Accuracy:", acc_fs)

# 7. Graph 1: Accuracy Comparison

plt.figure()
```

```
plt.bar(["PCA", "Feature Selection"], [acc_pca, acc_fs])
```

```
plt.title("Accuracy Comparison")
```

```
plt.ylabel("Accuracy")
```

```
plt.show()
```

```
# 8. Graph 2: PCA Explained Variance
```

```
plt.figure()
```

```
plt.plot(np.cumsum(pca.explained_variance_ratio_))
```

```
plt.title("PCA Explained Variance")
```

```
plt.xlabel("Number of Components")
```

```
plt.ylabel("Cumulative Variance")
```

```
plt.show()
```

